Machine learning Lab Observation

ITA0614 - Machine Learning

Experiment 6

Write a program to implement Naive Bayes algorithm in Python and display the results using confusion matrix and accuracy.

Name : Variganji Sumanth Babu

Reg.No : 192524447

Problem 1: Iris Flower Classification

Aim

To implement Gaussian Naive Bayes and classify iris flowers using four measured features, then evaluate the predictions with a confusion matrix and accuracy.

Algorithm

1. Store the flower measurements and species labels in a data list.
2. Separate the four features (X) from the target labels (y).
3. Split the data into stratified training and testing sets.
4. Train GaussianNB on the training set and predict the test labels.
5. Display the confusion matrix and test accuracy.

Naive Bayes: $P(C|X) \propto P(C) \times \prod P(x_i|C)$; accuracy = correct predictions / total test predictions.

Program:

nb_a.py - Python 3

```python
# Gaussian Naive Bayes: Iris flower classification
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

data = [
    [5.1,3.5,1.4,.2,"Setosa"], [4.9,3.0,1.4,.2,"Setosa"],
    [4.7,3.2,1.3,.2,"Setosa"], [5.0,3.6,1.4,.2,"Setosa"],
    [5.4,3.9,1.7,.4,"Setosa"], [7.0,3.2,4.7,1.4,"Versicolor"],
    [6.4,3.2,4.5,1.5,"Versicolor"], [6.9,3.1,4.9,1.5,"Versicolor"],
    [5.5,2.3,4.0,1.3,"Versicolor"], [6.5,2.8,4.6,1.5,"Versicolor"],
    [6.3,3.3,6.0,2.5,"Virginica"], [5.8,2.7,5.1,1.9,"Virginica"],
    [7.1,3.0,5.9,2.1,"Virginica"], [6.5,3.0,5.8,2.2,"Virginica"],
    [6.7,3.3,5.7,2.1,"Virginica"]]
X, y = [r[:4] for r in data], [r[4] for r in data]
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=.33, random_state=7, stratify=y)
model = GaussianNB().fit(Xtr, ytr)
pred = model.predict(Xte)
labels = ["Setosa", "Versicolor", "Virginica"]
print("Train / test samples:", len(ytr), "/", len(yte))
print("Class order:", labels)
print("Confusion matrix:\n", confusion_matrix(yte, pred, labels=labels))
print("Accuracy: {:.2%}".format(accuracy_score(yte, pred)))
```

Input

Iris data set: 15 illustrative flower records with Sepal Length, Sepal Width, Petal Length and Petal Width (cm). The target class is Species (Setosa, Versicolor or Virginica). A stratified train-test split is used.

Sepal L.	Sepal W.	Petal L.	Petal W.	Species
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa
4.7	3.2	1.3	0.2	Setosa
5.0	3.6	1.4	0.2	Setosa
5.4	3.9	1.7	0.4	Setosa
7.0	3.2	4.7	1.4	Versicolor
6.4	3.2	4.5	1.5	Versicolor
6.9	3.1	4.9	1.5	Versicolor
5.5	2.3	4.0	1.3	Versicolor
6.5	2.8	4.6	1.5	Versicolor
6.3	3.3	6.0	2.5	Virginica
5.8	2.7	5.1	1.9	Virginica
7.1	3.0	5.9	2.1	Virginica
6.5	3.0	5.8	2.2	Virginica
6.7	3.3	5.7	2.1	Virginica

Output

Terminal - Output

```
$ python nb_a.py
Train / test samples: 10 / 5
Class order: ['Setosa', 'Versicolor', 'Virginica']
Confusion matrix:
 [[1 1 0]
 [0 2 0]
 [0 0 1]]
Accuracy: 80.00%
```

Result

The Naive Bayes classifier was trained on the training subset and evaluated on unseen records. The confusion matrix above reports the number of correct and incorrect predictions in each class. The measured test accuracy for this run is 80.00%. The data are illustrative and intended for demonstrating the algorithm.

Problem 2: Student Pass Prediction

Aim

To predict whether a student passes from study hours, attendance and assignment score using Gaussian Naive Bayes, and report the confusion matrix and accuracy.

Algorithm

1. Prepare the numeric student features and Pass/Fail labels.
2. Split the records into stratified training and testing subsets.
3. Fit a Gaussian Naive Bayes classifier on the training records.
4. Predict Pass/Fail labels for unseen test records.
5. Print the confusion matrix and calculate the test accuracy.

Naive Bayes: $P(C|X) \propto P(C) \times \prod P(x_i|C)$; accuracy = correct predictions / total test predictions.

Program:

nb_b.py - Python 3

```python
# Gaussian Naive Bayes: student pass prediction
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

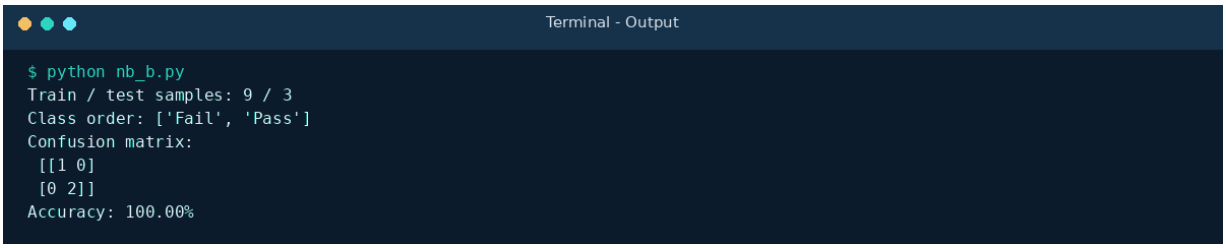
data = [
    [2,40,45,"Fail"], [3,50,50,"Fail"], [4,45,50,"Fail"],
    [4.5,55,55,"Fail"], [4,60,58,"Fail"], [5,62,60,"Fail"],
    [5.5,70,65,"Pass"], [6,75,72,"Pass"], [7,80,75,"Pass"],
    [8,88,80,"Pass"], [6.5,70,80,"Pass"], [7.5,85,85,"Pass"]]
X, y = [r[:3] for r in data], [r[3] for r in data]
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=.25, random_state=11, stratify=y)
model = GaussianNB().fit(Xtr, ytr)
pred = model.predict(Xte)
labels = ["Fail", "Pass"]
print("Train / test samples:", len(ytr), "/", len(yte))
print("Class order:", labels)
print("Confusion matrix:\n", confusion_matrix(yte, pred, labels=labels))
print("Accuracy: {:.2%}".format(accuracy_score(yte, pred)))
```

Input

Student data set: 12 illustrative records described by Study Hours, Attendance (%) and Assignment Score (out of 100). The target Result is Pass or Fail. The table shows the source records used for a stratified train-test split.

Study hrs	Attendance %	Assignment	Result
2	40	45	Fail
3	50	50	Fail
4	45	50	Fail
4.5	55	55	Fail
4	60	58	Fail
5	62	60	Fail
5.5	70	65	Pass
6	75	72	Pass
7	80	75	Pass
8	88	80	Pass
6.5	70	80	Pass
7.5	85	85	Pass

Output



Result

The Naive Bayes classifier was trained on the training subset and evaluated on unseen records. The confusion matrix above reports the number of correct and incorrect predictions in each class. The measured test accuracy for this run is 100.00%. The data are illustrative and intended for demonstrating the algorithm.

Problem 3: Bank Loan Approval

Aim

To use Gaussian Naive Bayes to classify loan applications from income, credit score and debt ratio, then evaluate the model using a confusion matrix and accuracy.

Algorithm

1. Store the loan attributes and approval labels.
2. Separate the numeric predictors from the target class.
3. Create stratified training and test sets.
4. Train Gaussian Naive Bayes and predict the test applications.
5. Display the confusion matrix and the classification accuracy.

Naive Bayes: $P(C|X) \propto P(C) \times \prod P(x_i|C)$; accuracy = correct predictions / total test predictions.

Program:

nb_c.py - Python 3

```
# Gaussian Naive Bayes: bank loan approval
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

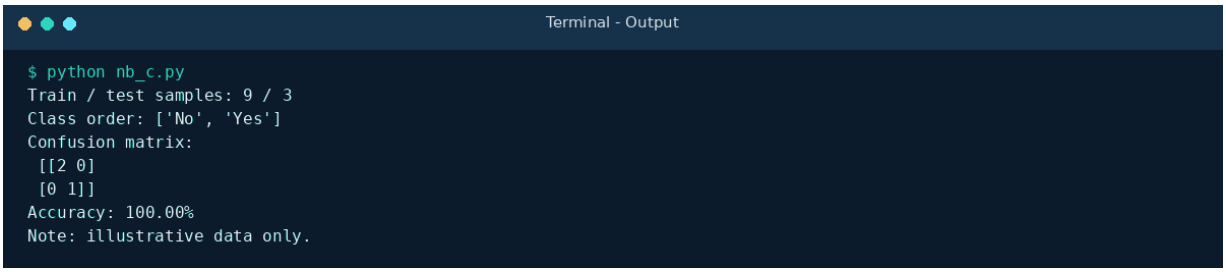
data = [
    [25,520,.70,"No"], [30,560,.65,"No"], [35,600,.55,"No"],
    [40,610,.50,"No"], [45,620,.45,"No"], [48,630,.42,"No"],
    [50,650,.35,"Yes"], [55,680,.30,"Yes"], [65,700,.25,"Yes"],
    [75,730,.20,"Yes"], [85,760,.15,"Yes"], [95,780,.10,"Yes"]]
X, y = [r[:3] for r in data], [r[3] for r in data]
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=.25, random_state=9, stratify=y)
model = GaussianNB().fit(Xtr, ytr)
pred = model.predict(Xte)
labels = ["No", "Yes"]
print("Train / test samples:", len(ytr), "/", len(yte))
print("Class order:", labels)
print("Confusion matrix:\n", confusion_matrix(yte, pred, labels=labels))
print("Accuracy: {:.2%}".format(accuracy_score(yte, pred)))
print("Note: illustrative data only.")
```

Input

Bank loan data set: 12 illustrative applications with Income (thousands), Credit Score and Debt Ratio. The target Approved is Yes or No. The dataset is a classroom example and is not a real lending policy.

Income (k)	Credit score	Debt ratio	Approved
25	520	0.70	No
30	560	0.65	No
35	600	0.55	No
40	610	0.50	No
45	620	0.45	No
48	630	0.42	No
50	650	0.35	Yes
55	680	0.30	Yes
65	700	0.25	Yes
75	730	0.20	Yes
85	760	0.15	Yes
95	780	0.10	Yes

Output



Result

The Naive Bayes classifier was trained on the training subset and evaluated on unseen records. The confusion matrix above reports the number of correct and incorrect predictions in each class. The measured test accuracy for this run is 100.00%. The data are illustrative and intended for demonstrating the algorithm.

Problem 4: SMS Spam Detection

Aim

To classify short SMS messages as Ham or Spam using text counts and Multinomial Naive Bayes, then display the confusion matrix and accuracy.

Algorithm

1. Store SMS messages and their Ham/Spam labels.
2. Split the messages into stratified training and test sets.
3. Convert the training messages into word-count vectors.
4. Train Multinomial Naive Bayes and classify test messages.
5. Print the confusion matrix and test accuracy.

Naive Bayes: $P(C|X) \propto P(C) \times \prod P(x_i|C)$; accuracy = correct predictions / total test predictions.

Program:

nb_d.py - Python 3

```python
# Multinomial Naive Bayes: SMS spam detection
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import confusion_matrix, accuracy_score

messages = [
 "Hey, are we meeting today?", "Can you send the notes?",
 "Please call me after class.", "I will reach in ten minutes.",
 "Are you coming to college?", "Thanks for helping me.",
 "Lets study for the exam.", "Your assignment looks good.",
 "Win a free prize now!", "Claim your cash reward today.",
 "Congratulations, you won a lottery.", "Urgent: claim your free gift.",
 "Get cheap loans, click now.", "You have won free tickets.",
 "Exclusive offer: earn cash fast.", "Text WIN to claim prize."]
y = ["Ham"] * 8 + ["Spam"] * 8
Xtr, Xte, ytr, yte = train_test_split(
    messages, y, test_size=.25, random_state=5, stratify=y)
vec = CountVectorizer()
Xtrain = vec.fit_transform(Xtr)
model = MultinomialNB().fit(Xtrain, ytr)
pred = model.predict(vec.transform(Xte))
labels = ["Ham", "Spam"]
print("Train / test messages:", len(ytr), "/", len(yte))
print("Class order:", labels)
print("Confusion matrix:\n", confusion_matrix(yte, pred, labels=labels))
print("Accuracy: {:.2%}".format(accuracy_score(yte, pred)))
```

Input

SMS dataset: 16 illustrative messages labelled Ham (legitimate) or Spam (unwanted). CountVectorizer converts message words into counts for Multinomial Naive Bayes. The table contains all records used in the stratified split.

SMS message	Class
Hey, are we meeting today?	Ham
Can you send the notes?	Ham
Please call me after class.	Ham
I will reach in ten minutes.	Ham
Are you coming to college?	Ham
Thanks for helping me.	Ham
Lets study for the exam.	Ham
Your assignment looks good.	Ham
Win a free prize now!	Spam
Claim your cash reward today.	Spam
Congratulations, you won a lottery.	Spam
Urgent: claim your free gift.	Spam
Get cheap loans, click now.	Spam
You have won free tickets.	Spam
Exclusive offer: earn cash fast.	Spam
Text WIN to claim prize.	Spam

Output



Result

The Naive Bayes classifier was trained on the training subset and evaluated on unseen records. The confusion matrix above reports the number of correct and incorrect predictions in each class. The measured test accuracy for this run is 100.00%. The data are illustrative and intended for demonstrating the algorithm.

Problem 5: Movie Review Sentiment

Aim

To classify short movie reviews as Positive or Negative using word counts and Multinomial Naive Bayes, and evaluate the predictions with a confusion matrix and accuracy.

Algorithm

1. Store short reviews and their Positive/Negative labels.
2. Split the reviews into stratified training and testing sets.
3. Convert the training text into word-count vectors.
4. Train Multinomial Naive Bayes and predict test sentiment.
5. Display the confusion matrix and calculate accuracy.

Naive Bayes: $P(C|X) \propto P(C) \times \prod P(x_i|C)$; accuracy = correct predictions / total test predictions.

Program:

nb_e.py - Python 3

```python
# Multinomial Naive Bayes: movie review sentiment
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import confusion_matrix, accuracy_score

reviews = [
 "excellent great wonderful film", "excellent amazing lovely movie",
 "wonderful excellent acting", "great amazing story",
 "lovely wonderful scenes", "excellent great movie",
 "amazing wonderful performance", "great excellent soundtrack",
 "awful terrible boring film", "awful horrible dull movie",
 "terrible boring acting", "poor horrible story",
 "boring dull scenes", "awful terrible movie",
 "horrible boring performance", "terrible poor soundtrack"]
y = ["Positive"] * 8 + ["Negative"] * 8
Xtr, Xte, ytr, yte = train_test_split(
    reviews, y, test_size=.25, random_state=13, stratify=y)
vec = CountVectorizer()
model = MultinomialNB().fit(vec.fit_transform(Xtr), ytr)
pred = model.predict(vec.transform(Xte))
labels = ["Negative", "Positive"]
print("Train / test reviews:", len(ytr), "/", len(yte))
print("Class order:", labels)
print("Confusion matrix:\n", confusion_matrix(yte, pred, labels=labels))
print("Accuracy: {:.2%}".format(accuracy_score(yte, pred)))
```

Input

Movie-review data set: 16 illustrative short reviews labelled Positive or Negative. CountVectorizer converts the review text into word counts, which are classified using Multinomial Naive Bayes.

Movie review	Sentiment
Excellent great wonderful film	Positive
Excellent amazing lovely movie	Positive
Wonderful excellent acting	Positive
Great amazing story	Positive
Lovely wonderful scenes	Positive
Excellent great movie	Positive
Amazing wonderful performance	Positive
Great excellent soundtrack	Positive
Awful terrible boring film	Negative
Awful horrible dull movie	Negative
Terrible boring acting	Negative
Poor horrible story	Negative
Boring dull scenes	Negative
Awful terrible movie	Negative
Horrible boring performance	Negative
Terrible poor soundtrack	Negative

Output



Result

The Naive Bayes classifier was trained on the training subset and evaluated on unseen records. The confusion matrix above reports the number of correct and incorrect predictions in each class. The measured test accuracy for this run is 100.00%. The data are illustrative and intended for demonstrating the algorithm.